

풀이 및 채점기준 (Version 2)

전자전기컴퓨터공학부 1학년 2021년 1학기 C프로그래밍 3반 (40121-3) 중간시험
(2021년 4월 22일 목요일 오후 1시~2시 50분; 총5쪽 5문제, 총점 100점)
(오픈북/온라인 시험)

백지 또는 전자패드에 반드시 수기로 답안을 작성할 것.

우선 “2021년 1학기 C프로그래밍 3반 중간시험”/학번/성명 기재 후, 풀이와 답을 기재.

시험 종료 시, 답지를 촬영하거나 스캔하여 jpg 또는 pdf 파일로 만든 후, 하나의 zip 파일에 담아

파일명을 “CP3mid(이름)(학번 마지막 자리 수)(예: CP3mid홍길동9)”으로 하고, 이를

반드시 학교 이메일 계정을 사용하여 아래 강사 이메일 주소로 송부.

김용한 교수(yhkim@uos.ac.kr) (조교에게는 송부하지 말 것)

아래 모든 소스 프로그램은 인텔 CPU를 탑재한 컴퓨터의 윈도우10 OS 상에서 Visual Studio 2019
Debug 모드 및 x86 모드를 사용하여 빌드하고 실행하였다.

문제 1. (16점) 아래 소스 프로그램에 대한 실행 예(아래 우하단에 나와 있음)를 보고 소스 프로그램에서 가려진 부분
의 내용(정수)과 출력 부분에서 가려진 내용(정수)을 문제 번호와 함께 써라.

```
#include <stdio.h>

int add1(int), add2(int), add3(int);
int num =         ; 문제 1-1

void main(void)
{
    register int num = 10;
    int line = 1;

    printf("LINE %02d: %d\n", line++, add1(num) + add2(num));
    printf("LINE %02d: %d\n", line++, add3(add2(num)));
    {
        int num = 20;

        printf("LINE %02d: %d\n", line++, num = add2(num));
    }
    printf("LINE %02d: %d\n", line++, add3(num));
}

int add1(int num)
{
    num += 30;
    return num;
}

int add2(int val)
{
    static int num =         ; 문제 1-2

    num += val;
    return num;
}

int add3(int val)
{
    num += add2(val);
    return num;
}
```

Microsoft Visual Studio 디버그 콘솔

```
LINE 01: 77
LINE 02: 153
LINE 03:          문제 1-3
LINE 04:          문제 1-4

I:\#00-강의\#0-강의-2021-1\#1-1 학년 C 프
니다<코드: 0개>.
이 창을 닫으려면 아무 키나 누르세요..
```

문제 1 끝.

답:

```
Microsoft Visual Studio 디버그 콘솔
LINE 01: 77
LINE 02: 153
LINE 03: 114
LINE 04: 277

I:W00-강의W0-강의-2021-1W1-1학년 C프
니다<코드: 0개>.
이 창을 닫으려면 아무 키나 누르세요..
```

문제 1-1의 답: **59**

문제 1-2의 답: **27**

문제 1-3의 답: **114**

문제 1-4의 답: **277**

채점 기준:

각 소문항 당 4점 배점. 부분 점수 없음

문제 1-1의 답과 문제 1-2의 답을 각기 x 와 y 라 하자. LINE 01의 경우, $\text{add1}(\text{num})$ 의 반환값이 $10+30=40$ 이고, $\text{add2}(\text{num})$ 의 반환값이 $10+y$ 이므로, 그 합은 $y+50$ 이 된다. 이를 출력값과 같다고 놓으면 $y+50=77$ 이 된다. 따라서 문제 1-2의 답은 $y=77-50=27$ 이다. 이 문장을 수행하고 나면, add2 의 static num 값은 37이 된다. (아래 표 참고 요망)

LINE 02의 경우, $\text{add2}(\text{num})$ 의 반환값은 $10+37=47$ 이고(이후 add2 의 static num 값이 47임), add3 함수 내에서 $\text{add2}(47)$ 이 호출되므로 그 반환값은 $47+47=94$ 가 되고(이후 add2 의 static num 값은 94임), global num 값은 $x+94$ 가 된다. 이 값을 출력값과 같다고 놓으면 $x+94=153$ 이 되어, 문제 1-1의 답은 $x=153-94=59$ 가 된다. (아래 표 참고 요망)

LINE 03의 경우, printf 문 내의 $\text{add2}(\text{num})$ 에서 num은 그 문장 바로 위에 선언된 num을 참조하므로, $\text{add2}(\text{num})$ 은 실제 $\text{add2}(20)$ 이고 그 반환값은 $94+20=114$ 이며(이후 add2 의 static num 값은 114임), 이 값은 해당 printf 문이 속한 중괄호 내의 local 변수 num에 저장된다. 따라서 문제 1-3의 답은 **114**이다. 중괄호 내의 local 변수 num은 중괄호를 빠져나올 때 그 메모리가 해제된다.

LINE 04의 경우, printf 문 내의 $\text{add3}(\text{num})$ 에서 num은 main 함수의 local 변수인 num의 값이므로, $\text{add3}(\text{num})$ 은 실제 $\text{add3}(10)$ 이다. add3 함수 내의 $\text{add2}(\text{val})$ 의 반환값은 $114+10=124$ 가 되고(이후 add2 의 static num 값은 124임), global num은 $153+124=277$ 이 되어 이 값이 출력된다 (아래 표 참고 요망) 따라서 문제 1-4의 답은 **277**이다.

	main의 local num	global num	add2의 static num
초기값	10	73	24
LINE 01 출력 후	10	73	37
LINE 02 출력 후	10	153	94
LINE 03 출력 후	10	153	114
LINE 04 출력 후	10	277	124

문제 1 풀이 끝.

문제 2. (16점) 아래 소스 프로그램에 대한 실행 예(아래 우하단에 나와 있음)를 보고 소스 프로그램에서 가려진 부분의 내용(두 자리의 수임)을 문제 번호와 함께 써라.

```
#include <stdio.h>

int funcA(int num)
{
    return ~num + 1;
}

int funcB(int num)
{
    return ~(num - 1);
}

int funcC(int num)
{
    int mask = 1;

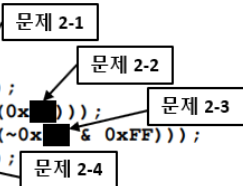
    do {
        if (num & mask)
            break;
        mask <<= 1;
    } while (mask);
    for (mask <<= 1; mask; mask <<= 1)
        num ^= mask;

    return num;
}

void showResult(int num)
{
    static int i = 0;

    printf("LINE %02d: %4d\n", ++i, num);
}

void main(void)
{
    showResult(funcA(0x[가려진 값]));
    showResult(funcA(funcA(0x[가려진 값])));
    showResult(funcB(funcB(~0x[가려진 값] & 0xFF)));
    showResult(funcC(0x[가려진 값]));
}
```



Microsoft Visual Studio 디버그 콘솔

```
LINE 01: -231
LINE 02: 195
LINE 03: 192
LINE 04: -165

I:\w00-강의\w0-강의-2021-1\w1-1학년 C 프
니다<코드: 0개>.
이 창을 닫으려면 아무 키나 누르세요.
```

문제 2 끝.

답:

문제 2-1의 답: **E7**

문제 2-2의 답: **C3**

문제 2-3의 답: **3F**

문제 2-4의 답: **A5**

채점 기준:

각 소문항 당 4점 배점. 답에 포함된 알파벳은 대소문자 구분 없이 글자가 맞으면 맞는 답임. 문제 2-3의 경우, XXXXXX1F(여기서 X부분은 최대 6개이며 각 X는 16진수 한 자리 숫자)라고 답한 경우도 감점하지 않음. 다른 부분 점수 없음

풀이:

어떤 양의 정수 x 를 $-x$ 로 바꾸려면 2의 보수를 취하면 된다. 만약 어떤 음의 정수 $-x$ (x 는 양수)를 양의 정수 x 로 바꾸려면 역시 2의 보수를 취하면 된다. 즉 $-(-x)=x$ 가 된다. 어떤 수 x 의 2의 보수 y 란 것은 이진수로 x 와 y 를 표현한 후 이진법에 의해 $x+y$ 를 하였을 때 0(모든 비트가 0가 되며,

MSB(Most Significant Bit, 최상위 비트)에서 발생하는 올림수 1은 나타낼 방법이 없으므로 삭제됨) 이 되는 수이므로, x 가 양수이든 음수이든 상관없이 그 2의 보수를 취하면 x 의 부호를 바꿀 수 있다.

2의 보수를 취하는 방법으로서 아래 세 가지 방법은 동등하다.

- (1) 1의 보수를 취한 후(모든 비트들을 반전시킨 후), 그 결과에 1을 더한다.
- (2) 1을 뺀 후, 그 결과에 1의 보수를 취한다(모든 비트들을 반전시킨다).
- (3) LSB(Least Significant Bit, 최하위 비트)부터 MSB 측으로 한 비트씩 읽어 내어, 처음으로 만나는 1까지의 비트들은 그 값을 그냥 두고, 그 다음 비트부터 MSB까지의 비트들을 반전시킨다.

(2)의 방법이 (1)의 방법과 동등함을 세 가지 경우로 나누어 보이고자 한다. 즉 (2)의 방법에 의해서도 (1)의 방법과 마찬가지로 정수의 부호를 바꿀 수 있음을 보이고자 한다. 이를 위해 주어진 수가 양수인 경우, 음수인 경우, 0인 경우의 세 가지 경우로 나누어 생각해 보자.

- (i) 어떤 양의 정수 x 가 있다고 할 때, 1을 뺀 후, 2의 보수를 취하면 $-(x-1)$ 이 될 것이다. 즉 $-x+1$ 이 된다는 것이므로, (2)의 방법에 의해, x 로부터 1을 뺀 후 2의 보수를 취하는 대신 1의 보수만 취하면 즉, 마지막에 더해 주는 1을 더하지 않으면(이는 1을 뺀 후 2의 보수를 취해 얻은 결과에서 1을 빼는 것과 같으므로), $(-x+1)-1=-x$ 가 된다. 즉 x 의 부호가 바뀐다.
- (ii) 또 어떤 음의 정수 $-y$ (y 는 양수)가 있다고 할 때, 1을 뺀 후, 2의 보수를 취하면 $-(-y-1)$ 이 될 것이다. 즉 $y+1$ 이 된다는 것이므로, 2의 보수를 취하는 대신 1의 보수만 취하면(즉 2의 보수를 취하는 과정에서 마지막에 1을 더하는 과정을 생략하면) $(y+1)-1=y$ 가 된다. 즉 $-y$ 가 y 로 되어 부호가 바뀐다.
- (iii) 만약 주어진 수가 0이라면, 1을 뺀 후, 2의 보수를 취하면, $-(-1)$ 이 될 것이다. 즉 1이 된다는 것이므로, 주어진 수 0으로부터 1을 뺀 후 2의 보수를 취하는 대신 1의 보수만 취하면(즉 2의 보수를 취하는 과정에서 마지막에 1을 더하는 과정을 생략하면) $1-1=0$ 이 된다. 즉 0은 0으로 되므로 부호가 바뀌는 것과 상관이 없다.

따라서 (2)의 방법은 (1)의 방법과 동등하다.

또 (3)의 방법이 (1)의 방법과 동등한 이유는 다음과 같다. 2의 보수 형태로 표현된 정수가 주어졌다고 하고, LSB부터 읽어서 n 번째 비트에서 처음으로 1이 등장한다고 하자. (1)의 방법에 따라서 이 정수에 대한 2의 보수를 구하기 위해, 이 수에 대해 1의 보수를 취하면 LSB부터 $n-1$ 번째 비트까지의 비트들은 모두 1, 또 n 번째 비트는 0으로 바뀌고 그 결과에 1을 더하면 LSB부터 $n-1$ 번째 비트까지는 모두 0으로 바뀌고 n 번째 비트(현재 값이 0)에 올림수 1을 더해야 한다. 따라서 주어진 정수의 2의 보수는 LSB부터 $n-1$ 번째 비트까지는 모두 0이고 n 번째 비트는 1이며, $n+1$ 번째 비트부터 MSB까지의 비트는 원래 비트를 반전시킨 값을 갖게 된다. 따라서 어떤 정수의 2의 보수를 취하는 한 가지 방법은 LSB부터 MSB 측으로 한 비트씩 읽어 내어, 처음으로 만나는 1까지의 비트들은 그 값을 그냥 두고, 그 다음 비트부터 MSB까지의 비트들을 반전시키는 것이다.

따라서 문제의 **funcA**, **funcB**, 그리고 **funcC**가 하는 일은 주어진 int 정수에 대해 2의 보수를 취하는 것 즉, 부호를 바꾸는 것으로서 동등하다.

main 함수의 첫 문장의 출력값은 -231이므로 funcA의 인자는 십진수로 $-(-231)=231$ 이며 이는 16진수로 E7이다. 문제2-1의 답은 **E7**이다.

main 함수의 두 번째 문장의 출력값은 195이므로 funcA의 인자는 십진수로 $-(-(195))=195$ 이며 이는 16진수로 C3이다. 문제 2-2의 답은 **C3**이다.

main 함수의 세 번째 문장의 출력값은 192이고 이는 16진수로 C0이므로 ~(문제 2-3 답) & 0xFF는 C0가 되어야 한다. 문제 2-3의 답은 int형의 4바이트 중 최하위 바이트의 비트들 중 상위 2비트만 0이라야 한다. 이 때 int형의 4바이트 중 상위 3바이트의 값은 결과에 영향을 끼치지 않는다. 따라서 문제 2-3의 답은 “XXXXXX1F(여기서 X부분은 don't-care 4bits로서 최대 6개이며 각 X는 16진수 한 자리 숫자)”라고 하는 것이 정확하겠으나, 문제의 지문에서 답이 2글자라 하였으므로, 문제 2-3의 답은 **3F**이다.

main 함수의 네 번째 문장의 출력값은 -165이므로 funcC의 인자는 십진수로 $-(-165)=165$ 이며 이는 16진수로 A5이다. 문제 2-4의 답은 **A5**이다.

문제 2 풀이 끝.

문제 3. (24점) 아래 소스 프로그램에 대한 실행 예에서 가려진 부분의 내용을 문제 번호와 함께 써라.

```
#include <stdio.h>

void setNum(int a[])
{
    static int num = 80;

    a[0] = num++; a[1] = ++num;
}

void swap1(int n1, int n2)
{
    int temp = n1;
    n1 = n2;
    n2 = temp;
}

void swap2(int *n1, int *n2)
{
    int* temp = n1;
    n1 = n2;
    n2 = temp;
}

void swap3(int *n1, int *n2)
{
    int temp = *n1;
    *n1 = *n2;
    *n2 = temp;
}

void swap4(int **n1, int **n2)
{
    int **temp = n1;
    n1 = n2;
    n2 = temp;
}

void swap5(int **n1, int **n2)
{
    int *temp = *n1;
    *n1 = *n2;
    *n2 = temp;
}

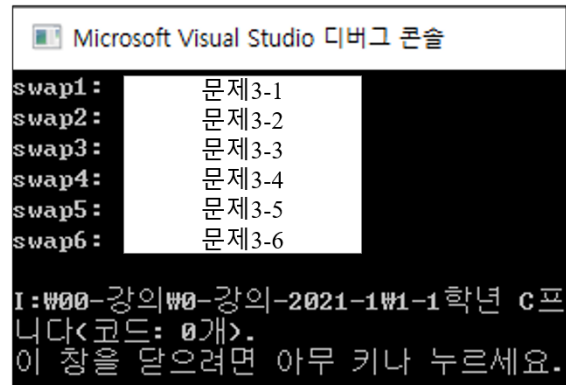
void swap6(int **n1, int **n2)
{
    int temp = **n1;
    **n1 = **n2;
    **n2 = temp;
}

void main(void)
{
    int num[] = { 0, 0 }, *p[] = { &num[0], &num[1] };

    setNum(num); swap1(num[0], num[1]);
    printf("swap1:  %d %d  %d %d\n", num[0], num[1], *p[0], *p[1]);

    setNum(num); swap2(p[0], p[1]);
    printf("swap2:  %d %d  %d %d\n", num[0], num[1], *p[0], *p[1]);
    setNum(num); swap3(p[0], p[1]);
    printf("swap3:  %d %d  %d %d\n", num[0], num[1], *p[0], *p[1]);

    setNum(num); swap4(p, p + 1);
    printf("swap4:  %d %d  %d %d\n", num[0], num[1], *p[0], *p[1]);
    setNum(num); swap5(p, p + 1);
    printf("swap5:  %d %d  %d %d\n", num[0], num[1], *p[0], *p[1]);
    setNum(num); swap6(p, p + 1);
    printf("swap6:  %d %d  %d %d\n", num[0], num[1], *p[0], *p[1]);
}
```



Microsoft Visual Studio 디버그 콘솔

```
swap1:      문제3-1
swap2:      문제3-2
swap3:      문제3-3
swap4:      문제3-4
swap5:      문제3-5
swap6:      문제3-6

I:\W00-강의\W0-강의-2021-1\W1-1학년 C프
니다<코드: 0개>.
이 창을 닫으려면 아무 키나 누르세요.
```

문제 3 끝.

답:

```
Microsoft Visual Studio 디버그 콘솔
swap1: 80 82 80 82
swap2: 82 84 82 84
swap3: 86 84 86 84
swap4: 86 88 86 88
swap5: 88 90 90 88
swap6: 92 90 90 92

I:W00-강의W0-강의-2021-1W1-1학년 C 프
니다<코드: 0개>.
이 창을 닫으려면 아무 키나 누르세요.
```

문제 3-1의 답: 80 82 80 82

문제 3-2의 답: 82 84 82 84

문제 3-3의 답: 86 84 86 84

문제 3-4의 답: 86 88 86 88

문제 3-5의 답: 88 90 90 88

문제 3-6의 답: 92 90 90 92

채점 기준:

각 값 당 1점(각 소문항 당 4점) 배점. 부분 점수 없음

풀이:

setNum 함수 내의 static 변수 num의 값은 처음 setNum 함수가 호출되었을 때 80으로 한 번 초기화되며, 이 후 setNum 함수가 다시 호출되었을 때 또 초기화되지는 않는다. 이는 일반적인 static 변수에 대한 초기화 과정이다. setNum 함수가 호출될 때마다, main 함수의 num 배열의 첫 요소에는 setNum 함수 내의 static 변수 num 값이 저장되고(a[0] = num++; 문장에서 후위 증가 연산자가 사용되었음에 유의), main 함수의 num 배열의 두 번째 요소에는 setNum 함수 내의 static 변수 num 값에서부터 둘 증가된 값이 저장된다(a[1] = ++num1; 문장에서 전위 증가 연산자가 사용되었음에 유의). 물론 setNum 함수가 호출될 때마다, static 변수 num 값은 2만큼 증가된다.

main 함수 내의 포인터 배열 p에는 main 함수의 num 배열의 첫 요소와 둘째 요소의 주소값이 저장된다.

swap1 함수 호출 직전의 setNum 함수 호출 결과, main 함수의 num 배열의 값은 80 82로 바뀐다. swap1 함수의 경우, call-by-value에 의한 함수이므로 이를 호출한 함수(여기서는 main 함수)의 변수 값을 변화시킬 수 없다. 따라서 문제 3-1의 답은 80 82 80 82이다. 이 출력 완료 시점에서 setNum 함수 내의 static 변수 num 값은 82이다.

swap2 함수 호출 직전의 setNum 함수 호출 결과, main 함수의 num 배열의 값은 82 84로 바뀐다. swap2 함수의 경우 call-by-reference에 의한 함수이긴 하지만, 내부적으로 매개변수의 값(이를 호출한 함수로부터 copy하여 넘겨받은 주소 값)만 서로 바꿀 뿐이므로 이를 호출한 함수(여기서는 main 함수)의 변수 값을 변화시킬 수 없다. 따라서 문제 3-2의 답은 82 84 82 84이다. 이 출력 완료 시점에서 setNum 함수 내의 static 변수 num 값은 84이다.

swap3 함수 호출 직전의 setNum 함수 호출 결과, main 함수의 num 배열의 값은 84 86으로 바뀐다. swap3 함수의 경우 call-by-reference에 의한 함수이고, 넘겨받은 주소지(main 함수의 배

열num의 두 요소의 주소지)에 저장된 값을 서로 바꾸므로, 이를 호출한 함수(여기서는 main 함수)의 변수 값(main 함수의 배열 num의 두 요소의 값)을 서로 바꾼다. 따라서 문제 3-3의 전반부의 답은 86 84이다. 문제 3-3의 후반부의 답의 경우, 배열 p에 저장된 주소값이 바뀐 것은 아니므로, p의 각 요소는 여전히 배열 num의 각 요소를 가리키고 있으므로, 전반부와 같은 값인 86 84가 답이 된다. 따라서 문제 3-3의 답은 **86 84 86 84**이다. 이 출력 완료 시점에서 setNum 함수 내의 static 변수 num 값은 86이다.

swap4 함수 호출 직전의 setNum 함수 호출 결과, main 함수의 num 배열의 값은 86 88로 바뀐다. swap4 함수의 경우 call-by-reference에 의한 함수처럼 보이긴 하지만, 내부적으로 매개변수의 값(이를 호출한 함수로부터 copy하여 넘겨받은 주소 값)만 서로 바꿀 뿐이므로 이를 호출한 함수(여기서는 main 함수)의 변수 값을 변화시킬 수 없다. 따라서 문제 3-4의 답은 **86 88 86 88**이다. 이 출력 완료 시점에서 setNum 함수 내의 static 변수 num 값은 88이다.

swap5 함수 호출 직전의 setNum 함수 호출 결과, main 함수의 num 배열의 값은 88 90으로 바뀐다. swap5 함수의 경우 call-by-reference에 의한 함수이고, 넘겨받은 주소지(main 함수의 배열 p의 두 요소의 주소지)에 저장된 값을 서로 바꾸므로, 이를 호출한 함수(여기서는 main 함수)의 변수 값(main 함수의 배열 p의 두 요소의 값)을 서로 바꾼다. 따라서 배열 p의 첫 요소는 num[1]를 가리키고, 두 번째 요소는 num[0]을 가리키는 것으로 바뀌어 있다. 그러므로 문제 3-5의 전반부의 답은 num[0]과 num[1]를 직접적으로 읽은 88 90이고, 후반부의 답은 배열 p를 통해 num[1]와 num[0]을 순차적으로 읽은 것이므로 90 88이다. 따라서 문제 3-5의 답은 **88 90 90 88**이다. 이 출력 완료 시점에서 setNum 함수 내의 static 변수 num 값은 90이다.

swap6 함수 호출 직전의 setNum 함수 호출 결과, main 함수의 num 배열의 값은 90 92로 바뀐다. swap6 함수의 경우 call-by-reference에 의한 함수이고, 넘겨받은 주소지(main 함수의 배열 p의 두 요소의 주소지)에 저장된 값 즉 main 함수의 배열 num의 두 요소의 주소지에 저장된 값을 서로 바꾸므로, 이를 호출한 함수(여기서는 main 함수)의 변수 값(main 함수의 배열 num의 두 요소의 값)을 서로 바꾼다. 문제 3-6의 전반부의 답은 값이 바뀐 num[0]과 num[1]를 직접적으로 읽은 92 90이다. 후반부의 답은, 현재 배열 p의 첫 요소는 num[1]을, 두 번째 요소는 num[0]을 가리키고 있는 상태(swap5 함수 호출 이후, 이러한 상태가 되었음에 주의)이므로, 배열 p를 통해 num[1]와 num[0]을 순차적으로 읽은 것이므로 그 출력 결과는 90 92이다. 따라서 따라서 문제 3-6의 답은 **92 90 90 92**이다.

문제 3 풀이 끝.

문제 4. (24점) 아래 소스 프로그램에 대한 실행 예에서 가려진 부분의 내용을 문제 번호와 함께 써라.

```
#include <stdio.h>

void main(void)
{
    char str[] = "\"What I want,\" said Mr. Gradgrind \"is Facts alone!\"";
    char str2[80] = { 0 };
    char *ps, *pd;
    int i, count = 0, line = 1, len = 0;

    for (i = 0; str[i] != 0; i++)
        if (str[i] == ' ')
            count++;
    printf("LINE %02d: %d %d \n", line++, count, len = ++i);

    printf("LINE %02d: %d %d %d %d %d %d %d %d %d \n",
        line++, ' ', '\\', '&', '\\', ',', '.', '0', '@', 'A');

    for (i = 0; str[i] != 0x2E; i++)
        if (str[i] < '0')
            str[i] = '0' + i % 10;
        else if (str[i] < 'a')
            str[i] += 'a' - 'A';
    for (i = 0; i < len; i++)
        if (str[i] == '9')
            break;
    str[i] = 0; len = i - 2;
    printf("LINE %02d: %s\n", line++, str);

    for (str[len /= 2] = i = 0; str[i]; i++)
        if (str[i] <= '9')
            str[i] = '0' + (str[i] - '0') * 2 % 10;
        else if (str[i] > 'Z') // Zz
            str[i] -= 'a' - 'A';
    printf("LINE %02d: %s\n", line++, str);

    str[len] = '?';
    ps = str + 3;
    ps[len * 2] = 0x40;
    printf("LINE %02d: %s\n", line++, str);

    for (i = len * 2 - 1; i >= 0; i--)
        if (str[i] == '@')
            break;
    for (ps = &str[i] + 1, pd = str2; *ps; )
        *pd++ = *ps++;
    printf("LINE %02d: %s\n", line++, str2);

    printf("LINE %02d: %c %c\n", line++, *(pd-- - 12), str2[6]);
}
```

Microsoft Visual Studio 디버그 콘솔

```

LINE 01: 문제 4-1
LINE 02: 32 34 38 39 44 46 48 64 65
LINE 03: 문제 4-2
LINE 04: 문제 4-3
LINE 05: 문제 4-4
LINE 06: 문제 4-5
LINE 07: 문제 4-6

I:\#00-강의\#0-강의-2021-1\#1-1학년 C프로그래밍\#4-중간시험\#3반 중간
니다<코드: 0개>.
이 창을 닫으려면 아무 키나 누르세요...
```

문제 4 끝.

답:

```
Microsoft Visual Studio 디버그 콘솔

LINE 01: 8 52
LINE 02: 32 34 38 39 44 46 48 64 65
LINE 03: 0what5i7want234said
LINE 04: 0WHAT0I4
LINE 05: 0WHAT0I4?ant234said@mr. Gradgrind "is Facts alone!"
LINE 06: mr. Gradgrind "is Facts alone!"
LINE 07: a a

I:W00-강의W0-강의-2021-1W1-1학년 C프로그래밍W4-중간시험W3반 중
니다<코드: 0개>.
이 창을 닫으려면 아무 키나 누르세요...
```

문제 4-1의 답: 8 52

문제 4-2의 답: 0what5i7want234said

문제 4-3의 답: 0WHAT0I4

문제 4-4의 답: 0WHAT0I4?ant234said@mr. Gradgrind "is Facts alone!"

문제 4-5의 답: mr. Gradgrind "is Facts alone!"

문제 4-6의 답: a a

채점 기준:

각 소문항 당 4점 배점.

문제 4-1 및 문제 4-6의 경우, 각 값 당 2점 배점.

문제 4-2의 경우, 첫 글자 0에 1점, 끝 부분 said에 1점, 나머지 부분에 2점 배점.

문제 4-4 경우, ?와 @의 위치가 맞을 경우, 사소하게 한 두 글자 틀린 것은 감점하지 않음.

문제 4-5 경우, 첫 부분 mr.를 제외한 다른 글자들 중 한 두 글자 사소하게 틀린 것은 감점하지 않음.

다른 부분 점수 없음

풀이:

"\What I want,\" said Mr. Gradgrind \"is Facts alone!\""에 포함된 빈 칸은 모두 8개이고, 눈에 보이는 글자(빈 칸 포함)는 51글자이고, 문자열 마지막에 널 문자가 하나 더 있으므로 str 배열의 글자 수는 모두 52바이트이다. 눈에 보이는 글자를 셀 때, \은 escape sequence로서 큰 따옴표 한 글자인 것으로 세야 한다. 따라서 52바이트가 str 배열에 저장되어 있으므로, LINE 01에서는 8 52가 출력된다. 따라서 문제 4-1의 답은 8 52이다.

LINE 02의 출력 내용으로부터, ' ', '\', '&', '\', ',', '.', '0', '@', 'A'의 값은 각기 32, 34, 38, 39, 44, 46, 48, 64, 65임을 알 수 있다.

이후 for (i = 0; str[i] != 0x2E; i++)문에서 str 배열의 글자 중 마침표(아스키 코드 값 0x2E 즉 10진수로 46) 직전까지의 글자 중, '0' 보다 그 코드 값이 작은 글자는 빈칸 즉, ' ', '\', '와 \이므로 이들은 str[i] = '0' + i % 10; 문장에 의해 모두 다른 글자로 바뀐다. 각 글자는 str 배열 내에서의 등장 위치 값을 10으로 나눈 나머지를 '0'에 더해 얻는 코드 값에 해당하는 글자로 바뀐다. 따라서 이렇게 바뀌는 글자는 7개이며, 차례로 0, 5, 7, 2, 3, 4, 9로 바뀐다. 또 '0' 보다 그 코드 값

이 작은 글자들을 제외하고 'a' 보다 그 코드 값이 작은 글자로서 마침표 직전까지 등장하는 글자는 대문자 W, I와 M이므로 이들은 각기 소문자 w, i 와 m으로 바뀐다. 이후 `for (i = 0; i < len; i++)` 문에 의해, 9로 바뀌었던 빈칸 위치에 널 문자(0 값)가 저장되어, str 배열에 저장된 문자열은 그곳에서 잘리게 된다. 따라서 문제 4-2의 답은 **0what5i7want234said**이다. 이 출력 직후 시점에서의 len 값은 17이다.

이후 `for (str[len /= 2] = i = 0; str[i]; i++)`에서 str 배열의 글자 수는 널 문자를 제외하고 8자인 것으로 str 배열에 저장된 문자열이 잘린다(len 값도 8로 바뀐다). 이 for문의 시행 결과, str 배열에 저장된 잘라진 문자열에 포함된 글자 중, '9' 보다 그 코드 값이 작거나 같은 글자는 아라비아 숫자에 해당하므로 이들이 다른 글자로 바뀌게 된다. 각 아라비아 숫자에 해당하는 글자는 그 수를 2배한 후 10으로 나눈 나머지에 해당하는 아라비아 숫자에 해당하는 글자로 바뀐다. '9' 보다 그 코드 값이 작거나 같은 글자들을 제외하고 'z' 보다 그 코드 값이 큰 글자는 알파벳 소문자인데, 이들은 `str[i] -= 'a' - 'A'`;문에 의해 모두 대문자로 바뀐다. 따라서 문제 4-3의 답은 **OWHAT0I4**이다.

이후 str 배열의 요소 값 중, 9번째 요소의 값인 널 문자가 '?'로 바뀌고, 20번째 요소의 값인 널 문자가 **0x40**(십진수로 64)에 해당하는 '@'로 바뀐다. 이로 인해 str 배열에 저장되어 있던 잘린 문자열은 잘리기 전 최초의 길이를 갖는 문자열로 복원된다. 따라서 문제 4-4의 답은

OWHAT0I4?ant234said@mr. Gradgrind "is Facts alone!"

이다. 단, len의 값은 이 과정을 모두 마친 지점에서 16으로 바뀐다.

이 후, str 배열에 저장된 문자열에서 32번째 요소(Gradgrind의 n)로부터 거꾸로 읽어 '@'인 첫 문자를 찾은 후, 방향을 바꿔 순방향으로 이 문자 다음 문자(큰 따옴표)부터 글자들을 복사하여 str2에 저장한다. 마지막 문자인 '\0'을 복사한 후, 그 다음 요소에 널 문자를 저장하여 문자열을 완성하지 않더라도, str2에 저장된 내용이 하나의 문자열이 되는 이유는 str2를 선언할 때, `char str2[80] = { 0 };`와 같이 초기화함으로써, 80개의 요소에 모두 0(널 문자)이 이미 저장되어 있기 때문이다. 따라서 문제 4-5의 답은

mr. Gradgrind "is Facts alone!"

이다.

상기 문자열 복사가 종료되었을 때, pd는 str2에 저장된 문자열의 마지막 부분에 있는 널 문자를 가리키고 있는 상태에 있다. 따라서 `*(pd-- - 12)`는 str2에 저장된 문자열의 내용 중에서 마지막에서 12번째 글자인 a에 해당한다(pd--는 후감소 연산임에 주의). 물론 `str2[6]`는 str2에 저장된 문자열의 내용 중에서 처음에서 7번째 글자인 a에 해당한다. 따라서 문제 4-6의 답은 **a a**이다.

문제 4 풀이 끝.

문제 5. (20점) 아래 소스 프로그램과 그 실행 예를 보고 아래 물음에 답하라.

```
#include <stdio.h>

void func(int num)
{
    int flag = 0;
    for ( [문제 5-1]; i >= 0; i--)
    {
        if ( [문제 5-2] )
        {
            printf("%d", 1);
            flag = 1;
        }
        else
        {
            if ( [문제 5-3] )
            {
                [문제 5-4];
            }
        }
        printf("\n");
    }
}

void main(void)
{
    int num;

    while (1)
    {
        printf("10진수 정수 입력: ");
        scanf("%d", &num);
        if (num == 0)
            break;
        func(num);
    }
}
```

문제:

문제 5-1 ~ 문제 5-4: 가려진 부분에 들어 갈 C 언어 표현(expression)을 각 문제 별로 답하라. 단, 문제 5-1의 경우, 프로그램이 platform-independent하게 되도록 답을 제시하라.

문제 5-5: 가려진 부분에서 사용자가 2글자를 입력하고 엔터 키를 쳤다. 이 2글자를 답하라.

문제 5 끝.

때 마다 num 변수의 1비트를 처리하므로 이 for문의 반복 횟수는 32회가 되어야 하는데, 이 for문의 조건식이 i가 0일 때에도 참이 되도록 나와 있으므로, 초기식에서의 초기값은 31이 되어야 한다. 또 for문의 인덱스 변수인 i를 선언한 곳이 없으므로, 이를 선언하는 부분도 포함되어야 한다. 따라서 문제 5-1의 답은 `int i = sizeof(int) * 8 - 1`이다. 문제의 지문에서 “프로그램이 platform-independet하게 되도록” 하라고 요구하였으므로, 이 프로그램이 CPU, OS, compiler 등의 종류에 무관하게 동작할 수 있도록 반드시 sizeof 연산자를 사용해야 한다.

func 함수 내의 for문의 반복영역에서 num 변수가 활용되는 부분이 나와 있지 않으므로 if~else문의 조건식에 num 변수가 개입되어 있어야 함을 알 수 있다. if~else문의 if 블록 내의 `printf("%d", 1);`로부터 if 블록에서는 1을 1개 출력하고 else 블록에서는 0을 1개 출력하는 역할을 해야 한다는 것을 알 수 있으므로, 이 if~else문의 조건식은 이번에 출력할 num의 1비트가 1이면 참이 되어야 함을 알 수 있다. 이와 같은 점을 모두 고려하면, 이 조건식에서 num의 검사 대상 비트 위치에만 1인 값을 만들어 num과 비트단위 AND하는 것으로 하면 소기의 목적을 거둘 수 있음을 알 수 있다. num의 검사 대상 비트 위치에만 1인 값을 만드는 한 가지 방법은 1을 i번 좌로 시프트하는 것이다. 따라서 문제 5-2의 답은 `num & 1 << i` 또는 `1 << i & num`이다.

상기 언급한 else 블록에서는 출력할 비트의 값이 leading 0s에 속하지 않을 때에만 0을 출력해야 하므로, 이를 위해 flag 변수가 사용되었음을 알 수 있다. 즉 flag는 0으로 초기화되었다가 처음으로 1값에 해당하는 비트를 출력하면 flag가 1로 변경되므로, 이 else 블록 내의 if문의 조건식은 flag가 1이면 참이 되어야 한다. 조건식의 값이 0이 아니면 참으로 간주되므로 flag 자체만 조건식에 넣어도 된다. 따라서 문제 5-3의 답은 `flag` 또는 `flag == 1`이 된다.

물론 문제 5-4의 답은 `printf("%d", 0)`이다.

문제의 지문의 출력 내용 중, 두 번째 줄을 보면 32비트 중 최상위 비트가 1로 출력되어 있으므로, 프로그램 사용자가 입력한 값은 음수이다. 따라서 이 출력 내용에 대한 2의 보수를 구하면, 즉 출력된 32비트를 모두 반전시킨 후 1을 더하면 프로그램 사용자가 입력한 값의 절대값을 얻는다. 그 결과는 9이므로 프로그램 사용자가 입력한 내용은 -9 두 글자이다. 따라서 문제 5-5의 답은 -9이다.

문제 5 풀이 끝. 전체 끝.